

Java Framework

1. **Spring Framework:** គឺជា Opensource Framework ដែលមានលក្ខណៈស្រាលនិងត្រូវបានប្រើប្រាស់យ៉ាងទូលំទូលាយសម្រាប់ការបង្កើតកម្មវិធីជាមួយនឹងភាសា Java ។
2. **Spring core:** គឺជាសមាសធាតុស្នូលរបស់ Spring Framework សម្រាប់ការបង្កើតនិងការគ្រប់គ្រង Object ។
 - វាមានមុខងារសំខាន់ៗ ២ គឺ Inversion of Control & Dependency Injection ។
3. **Spring MVC:** (Model View Controller): គឺប្រើសម្រាប់បង្កើត Web Application ដែលមានភាពបត់បែននិងងាយស្រួលកែតម្រូវការអនុវត្តតាមគោលគំនិត MVC Architecture ។
 - Architecture មាន ៣ ផ្នែកគឺ:
 - Model: ប្រើសម្រាប់ភ្ជាប់ទៅកាន់ Database
 - View: ប្រើសម្រាប់ Frontend
 - Controller: ប្រើសម្រាប់ភ្ជាប់ទំនាក់ទំនងរវាង View & Model
4. **Spring AOP:** (Aspect Oriented Programming): គឺជាមុខងារដែលអាចបំបែកកូដទៅជា Module តូចៗជាច្រើនទៅតាមអទិភាពនៃការងារនៅពេល Run time ។
5. **Spring DAO:** (Java Data Object): គឺជាសមាសធាតុដែលប្រើសម្រាប់ភ្ជាប់ទំនាក់ទំនងរវាង Database ជាមួយ Spring MVC ។
6. **Spring ORM:** (Object Relational Mapping): ជាសមាសធាតុដែលមានទំនួលខុសត្រូវក្នុងការចូលទៅប្រើប្រាស់និងរៀបចំទិន្នន័យជាមួយនឹង Data Framework មួយចំនួន Java Persistence API (JPA), Java Data Object (JDO), iBATIS SQL ជាដើម។
7. **Spring Context:** ជាអ្នកគ្រប់គ្រងដំណើរការនៅក្នុង Spring Project រួមមានការបង្កើត Object ការ Config Object & ការប្រមូលផ្តុំ Object នៅក្នុង class ដែលទទួលបានពី xml & java file ។
8. **Spring Boot:** ជាបច្ចេកវិទ្យាមួយដែលជួយអ្នកបង្កើតកម្មវិធីជាមួយនឹងភាសា Java មានភាពងាយស្រួលនិងរហ័សជាងមុន វាផ្តល់នូវមុខងារដែលអាចជួយឱ្យអ្នកចាប់ផ្តើមគម្រោងថ្មីបានយ៉ាងរហ័សនិងមិនចាំបាច់កំណត់រចនាសម្ព័ន្ធដូច្នេះទេ។
9. **IOC Container:** ជា Framework សម្រាប់ប្រើប្រាស់ Dependency ដោយស្វ័យប្រវត្តិ។
 - មាន ២ គឺ Bean factory & Application Context
 - មុខងារសំខាន់ៗ

- ដើម្បីបង្កើត Application class
- កំណត់ Object
- ប្រមូលផ្តុំ Dependency រវាង Object

10. Dependency Injection: ជាធាតុមូលដ្ឋានមួយនៅក្នុង Spring Container ដែលត្រូវបានប្រើដើម្បីធ្វើការហៅ Object មួយទៅក្នុង Object មួយទៀត។

11. POJO: (Plain Old Java Object): វាជាប្រភេទ Object ដើមដែលមិនមានដែនកំណត់ដោយទម្រង់ណាមួយនោះទេ។

12. Spring Bean: រាល់ Object ទាំងអស់នៅក្នុង Spring Framework ដែលយើងប្រកាន់តាមរយៈ Spring Container ។

13. Bean Life Cycle: ជាដំណាក់កាលមួយនៃ Bean ដែលគិតចាប់ពី Bean ត្រូវបានបង្កើត ប្រកាសប្រើប្រាស់ ការប្រើប្រាស់កំឡុង Run time នៃកម្មវិធីនិងចុងក្រោយវាត្រូវបានបញ្ចប់។

14. Spring Bean Scope: មាន scope ចំនួន ៥ ដូចខាងក្រោម

- Singleton: មានតែមួយប៉ុណ្ណោះដែលត្រូវបានបង្កើតសម្រាប់ Container នីមួយៗ។
- Prototype: Object Bean ថ្មីមួយនិងត្រូវបានបង្កើតគ្រប់ពេលដែល Bean មានការស្នើសុំប្រើប្រាស់
- Request: វាស្រដៀងទៅនឹង Protocol type scope ដែលវាត្រូវបានគេប្រើប្រាស់ជាមួយ Web Application
- Session: Object Bean ថ្មីនឹងត្រូវបានបង្កើតសម្រាប់ HTTP Session នីមួយៗដោយ Container
- Global Session: វាត្រូវបានគេប្រើដើម្បីបង្កើត Global Session Bean សម្រាប់ Portlet Application

15. @Required: ប្រើដើម្បីហៅ Setter Method របស់ Bean មកប្រើប្រាស់។

16. @Autowired: ប្រើដើម្បីហៅ Method មួយចំនួនដូចជា Setter, Getter & Container យកមកប្រើប្រាស់។

17. @Qualifier: ស្រដៀងទៅនឹង @Autowired ដែរប៉ុន្តែវាអាចលុបចោលនៅភាពយល់ច្រឡំមួយចំនួនសម្រាប់ការហៅ Bean ច្រើនជាងមួយ។

18. Lombok: ជា Library របស់ Java ដែលត្រូវបានយកបញ្ចូលដោយស្វ័យប្រវត្តិនៅក្នុង Editor & Build Tool សម្រាប់ផ្សំបំពាក់ Java របស់អ្នក។

19. Core Annotation in Lombok:

- @AllArgsConstructor: ដើម្បីបង្កើត Constructor ជាមួយគ្រប់ Parameter ទាំងអស់
- @NoArgsConstructor: ដើម្បីបង្កើត Constructor ដែលមិនមាន Parameter ឬអត់មាន Arguments

- @RequiredArgsConstructor: ដើម្បីបង្កើត Constructor ជាមួយនឹង field ឬ Parameter ដែលកំណត់ដោយ Developer (field ត្រូវមាន keyword final ឬ annotation @NonNull)
- @Data: ដើម្បីប្រមូលផ្តុំនូវ Annotation មួយចំនួនដូចជា:
 - @Getter: បង្កើត Getter Method សម្រាប់ Field ទាំងអស់
 - @Setter: បង្កើត Setter Method សម្រាប់ Field ទាំងអស់
 - @ToString: បង្កើត ToString Method
 - @EqualsAndHashCode: ដើម្បី Generate equals() & hashCode() Method

20. Swagger: គឺជា Open API Library សម្រាប់ការចនា Rest API ដែលវាបានផ្តល់នូវ Tools មួយចំនួនដូចជា Swagger editor, Swagger UI, Swagger Codegen ជាដើម។

21. Spring Data JPA: គឺជា Project មួយដែលត្រូវបានបញ្ចូលនៅក្នុង Spring Project ដើម្បីធ្វើការជាមួយប្រភពទិន្នន័យផ្សេងៗនិង Database។

22. Entity Mapping:

- Entity: ត្រូវបានគេស្គាល់ថា Object ដែលប្រើជាមួយនឹងប្រភពទិន្នន័យផ្សេងៗ
- Annotations: ដែលប្រើជាមួយ Data Spring JPA ដូចជា
 - @Entity: ដើម្បីបញ្ជាក់ class នោះជា Entity
 - @Table: ដើម្បីបញ្ជាក់ Table នោះប្រើសម្រាប់ Entity
 - @Id: ប្រើសម្រាប់កំណត់ថាជា Primary Key
 - @GeneratedValue: សម្រាប់ផ្តល់ជាការកំណត់លក្ខណៈផ្សេងៗទៀត Primary key
- Mapping Relationships:
 - @OneToOne: ប្រើសម្រាប់ទំនាក់ទំនង One to One
 - @OneToMany: ប្រើសម្រាប់ទំនាក់ទំនង One to Many
 - @ManyToMany: ប្រើសម្រាប់ទំនាក់ទំនង Many to Many
 - @ManyToOne: ប្រើសម្រាប់ទំនាក់ទំនង Many to One

23. Spring MVC Framework: អនុវត្តតាមគំរូនៃការចនា Model view Controller ដែលធ្វើការជុំវិញ Front Controller

24. Dispatcher Servlet: ប្រើសម្រាប់ទទួលខុសត្រូវនិងបញ្ជូនការស្នើរសុំទិន្នន័យ (HTTP Request) ទៅកាន់ Controller ដែលត្រូវនឹងការស្នើរសុំ។

25. **@Controller**: ជា Annotation ដែលប្រើសម្រាប់កំណត់ផ្នែក Class មួយជា Controller ដែលដំណើរការជាមួយ Webpage។
26. **@RequestMapping**: ជា Annotation ដែលប្រើសម្រាប់គូសផ្គុំនូវ Map Request ទៅកាន់ Method របស់ Spring Controller។
27. **Model**: ជាកន្លែងសម្រាប់ផ្ទុកទិន្នន័យរបស់កម្មវិធី។
28. **Controller**: ជាផ្នែកដែលទទួលខុសត្រូវលើ Business Logic នៃកម្មវិធី។
29. **View**: ប្រើសម្រាប់តំណាងឱ្យការផ្តល់ព័ត៌មាននៅក្នុងទម្រង់ជាក់លាក់មួយ ជាទូទៅអាចគេប្រើ JSP & JSPL សម្រាប់បង្ហាញជា View page។
30. **Front Controller**: នៅក្នុង Spring Web MVC Front Controller មានតួនាទីគ្រប់គ្រងនូវលំហូរនៃកម្មវិធីរបស់ Spring MVC។
31. **Thymeleaf**: គឺជាបច្ចេកវិទ្យាផ្នែក Frontend ដែលប្រើជាមួយ Spring Framework ដើម្បីបង្កើត Web Application ដែលវាមានសមត្ថភាពដំណើរការក្នុង HTML, XML, JavaScript & CSS ជាដើម។

❖ Swagger

@Configuration

```
public class OpenApiConfig {
    @Value("${project.openapi.dev-url}")
    private String devUrl;
    @Value("${project.openapi.prod-url}")
    private String prodUrl;
    @Bean
    public OpenAPI myOpenAPI(){
        Server devServer = new Server();
        devServer.setUrl(devUrl);
        devServer.setDescription("Server URL in Development environment");
        Server prodServer = new Server();
        prodServer.setUrl(prodUrl);
        prodServer.setDescription("Server URL in Production environment");
    }
}
```

```

Info info = new Info()
    .title("Libray Management API")
    .version("1.0")
    .description("This API exposes endpoints to manage library.");
return new OpenAPI().info(info).servers(List.of(devServer, prodServer));
}
}

```

❖ Controller

```

@Controller

public class UserController {

    @Autowired
    private UserService userService;

    // Display the home page with all users
    @GetMapping("/")
    public String viewHomePage(Model model) {
        model.addAttribute("ListUsers", userService.getAllUsers());
        return "index";
    }

    // Show the form to create a new user
    @GetMapping("/showNewUserForm")
    public String showNewUserForm(Model model) {
        User user = new User(); // Create a new user object
        model.addAttribute("user", user); // Add the user object to the model
        return "new_user"; // Return the view name
    }
}

```

```

// Handle the form submission to save a new user
@PostMapping("/saveUser")
public String saveUser(@ModelAttribute("user") User user) {
    userService.saveUsers(user); // Save the user to the database
    return "redirect:/"; // Redirect to the home page
}

// Show the form to update an existing user
@GetMapping("/showFormForUpdate/{id}")
public String showFormForUpdate(@PathVariable(value = "id") long id, Model model) {
    // Get the user from the database
    User user = userService.getUserById(id);
    model.addAttribute("user", user); // Add the user to the model
    return "update_user"; // Return the update form view
}

// Handle the form submission to update an existing user
@PostMapping("/updateUser")
public String updateUser(@ModelAttribute("user") User user) {
    userService.updateUser(user); // Update the user in the database
    return "redirect:/"; // Redirect to the home page
}

// Handle the delete user action
@GetMapping("/deleteUser/{id}")
public String deleteUserById(@PathVariable(value = "id") long id) {
    userService.deleteUserById(id); // Delete the user from the database
    return "redirect:/"; // Redirect to the home page
}
}

```

❖ API

```
@RestController
@RequestMapping("/api/users")
public class UserApiController {

    @Autowired
    private UserService userService;

    //Get all users
    @GetMapping
    public List<User> getAllUsers(){
        return userService.getAllUsers();
    }

    //Get user by id
    @GetMapping("/{id}")
    public ResponseEntity<User> getUserById(@PathVariable Long id){
        User user = userService.getUserById(id);
        if(user != null){
            return ResponseEntity.ok(user);
        }else{
            return ResponseEntity.notFound().build();
        }
    }

    //Create new User
    @PostMapping("/{id}")
    public User createUser(@RequestBody User user){
        userService.saveUsers(user);
        return user;
    }
}
```

```

//Update an existing user
@PutMapping("/{id}")
public ResponseEntity<User> updateUser(@PathVariable Long id,
    @RequestBody User userDetails){
    User user = userService.getUserById(id);
    if (user != null){
        user.setName(userDetails.getName());
        user.setEmail(userDetails.getEmail());
        user.setPassword(userDetails.getPassword());
        userService.saveUsers(user);
        return ResponseEntity.ok(user);
    }else{
        return ResponseEntity.notFound().build();
    }
}

//Delete a user
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteUser(@PathVariable Long id){
    User user = userService.getUserById(id);
    if(user != null){
        userService.deleteUserById(id);
        return ResponseEntity.noContent().build();
    }else{
        return ResponseEntity.notFound().build();
    }
}
}

```



```
@Entity
@Table(name="thymeleaf_tbl_user")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String password;
}
```

❖ Repository

```
@Repository
public interface UserRepository extends JpaRepository<User, Long>{
}
```

❖ Service

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    // Get all users
    public List<User> getAllUsers(){
        return userRepository.findAll();
    }

    // Save a new user
}
```

```

public void saveUsers(User user){
    userRepository.save(user);
}

// Get a user by ID
public User getUserById(Long id){
    return userRepository.findById(id).orElse(null);
}

// Delete a user by ID
public void deleteUserById(Long id){
    userRepository.deleteById(id);
}

// Update an existing user
public void updateUser(User user){
    userRepository.save(user); // This will update the user if it exists
}
}

```

Code ទាំងអស់គ្រាន់ជា Example ប៉ុណ្ណោះមិនអាចត្រូវនឹងការចេញប្រលងទេ